Block device analysis

Overview

- General concept
  - consider data at the lowest level of organization – blocks
  - blocks = fixed-size sequences of bytes
- Goals
  - provide independent means of reconstructing logical objects – ***FILESYSTEMS***, ***DATABASES***, etc.
    - avoids having to trust the running system
    - provides access to deallocated data
  - reconstruct higher level data objects, ***FILES***, in the absence / corruption of metadata due to
    - normal deallocation operations
    - system failures
    - attempts to hide data
  - provide low-level access to data storage
    - slack space, blocks marked as corrupt, etc.

Methods

- Reconstruction based on metadata
  - e.g., TSK tools understand FS metadata
- Block-level analysis
  - entropy → general statistical data classifications
  - block hashes → identification of *specific* content
  - raw data → binary string patterns (+ higher level rules)
- Data carving
  - recover (+ validate) logical data objects (files, typically) from the raw sequence of blocks
    - usually, no metadata
  - based on
    - knowledge of the specifics of data format
    - understanding of the details FS operations (allocation / deallocation)

Entropy

What Problem Are We Solving?

You have a 500 GB raw disk image.

- Where are the interesting regions?
- Is that blank space *really* blank — or is it an encrypted volume?
- That carved file — is it valid or garbage?

Entropy gives you a fast, first-pass answer.

Shannon's entropy formula

$$H = -\sum_{i=0}^{255} p\,(x_i)\log_2 p(x_i)$$

- $p(x_i)$: frequency of byte value i in the data
- Result is in **bits per byte**
- Range: **0.0** (all bytes identical) → **8.0** (perfectly uniform)

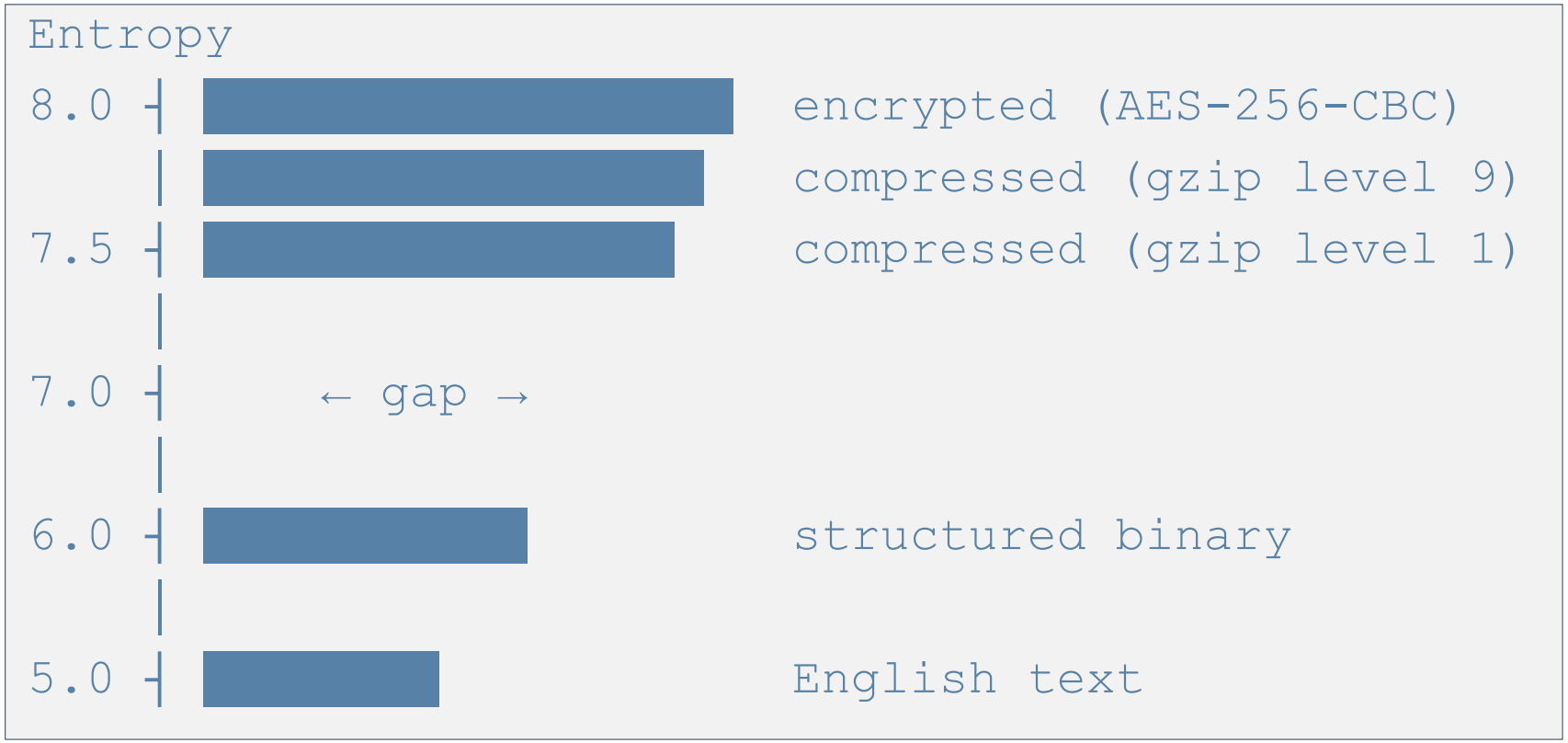
. . .

Think of it as: **"how surprised am I, on average, by the next byte?"**

Intuition check

Input	Entropy	Why
1 MB of `\x00`	`0.0`	Zero surprise — every byte is 0
`AAAAAABBCC`	`~1.30`	3 symbols, heavily skewed to A
`/dev/urandom` (1 MB)	`~7.999`	All 256 values equally likely

THE AMBIGUOUS ZONE: 7.5 – 8.0



Both compressed and encrypted data are **dense**

➔ entropy *alone* cannot tell them apart.

COMPRESSION ➔ HIGH ENTROPY

- A good compressor removes local redundancy.
  - input: English text at $H \approx 4.5$ with repeated patterns
  - output: dense bitstream at $H \approx 7.9$ — patterns are gone at the byte level
- But the compressed stream still has structure:
  - magic bytes (\x**1f**\x**8b** for ***gzip***)
  - block headers, Huffman tables, length fields
  - predictable framing
- Entropy is high, but the data is identifiable and parseable.

ENCRYPTION ➔ HIGH ENTROPY

- A good cipher produces output computationally indistinguishable from random.
  - No magic bytes (unless in an outer container like LUKS)
  - No internal structure
  - No block boundaries visible in the stream
  - Cannot be further compressed

➔ High entropy and no structure ➔ hallmark of encryption.

TAKEAWAY: ENTROPY IS NECESSARY BUT NOT SUFFICIENT

Low entropy? | Not encrypted | (text, zeroed, structured)

High entropy?

```
Could be compressed OR encrypted
Need more analysis:
  • magic bytes / binwalk
  • chi-square test
  • block alignment analysis
```

➔ Entropy is a **triage tool**, not a final answer.

Complementary tests

- Magic bytes / `file` / `binwalk`:
  - compressed formats have identifiable headers — encrypted blobs don't.
- Chi-square (χ^2) distribution test:
  - truly random data passes χ^2 uniformity
  - compressed data often has slight deviations most coding doesn't produce perfect uniformity.
- `ent` gives yields both
  - entropy *and* chi-square in one command.

Forensic Application: Disk Triage

◊ Slide a window across the raw image, compute entropy per block:

```
Offset (MB)

 0      50     100    150    200    250    300

 |------|▀▀▀▀▀▀|▬▬▬▬▬▬▬▬|------|▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀|

 zeroed   gzip    text     zeroed   encrypted?
```

In seconds we know ***where***

- » the **allocated,** potentially **interesting data** is
- » the **free/wiped space** is (skip it)
- » the **uniform high-entropy regions** are ➔ need further investigation

Forensic application: hidden volume detection

◊ A “wiped” drive should be all zeroes → entropy == 0.

Is this wiped?

➔ 200 GB region at $H \approx 7.999$ is **not** wiped.

Likely: TrueCrypt/VeraCrypt hidden volume, LUKS partition, or full-disk encryption.

Forensic application: steganography

- Image with hidden data in least-significant bits:
  - Overall entropy may barely change
  - LSB-only entropy jumps noticeably
- Extracting and measuring entropy of specific bit planes can reveal embedded payloads.

THE ent TOOL

```
$ ent sample.bin
Entropy = 7.999825 bits per byte.
  Optimum compression would reduce the size
   of this 1048576 byte file by 0 percent.
Chi square distribution for 1048576 samples is 233.45, and
   randomly would exceed this value 75.00 percent of the times.
Monte Carlo value for Pi is 3.14157362 (error 0.00 percent).
Serial correlation coefficient is -0.000146 (totally
uncorrelated).
```

One command → five statistical tests → no dependencies

READING `ent` OUTPUT

Metric	Interpretation
Entropy (bits/byte)	Information density (our main focus)
Chi-square	How close to uniform distribution
Compression %	Redundancy remaining
Serial correlation	Byte-to-byte dependence

PUTTING IT TOGETHER: THE TWO-TEST TRIAGE

```
                Entropy < 7.0?
               /               \
            YES                 NO
             |                   |
   Not encrypted        Chi-square %
   (text, binary,        in 10-90%?
    zeroed, etc.)       /          \
                     YES            NO
                      |              |
              Likely encrypted  Likely compressed
                                (or mixed content)
```

Neither test is definitive alone. Together they cover most cases.

Summary

1. **Entropy = bits of surprise per byte** — range 0.0 to 8.0
2. **Content types have characteristic bands** — powerful for fast triage
3. **High entropy is ambiguous** — compressed and encrypted overlap
   - always combine with structural analysis (magic bytes, χ^2, `binwalk`)
4. **Entropy is a triage tool**, not a classifier
   - fast first pass → then targeted deep analysis

Hands-on exercise

```
git clone https://gitlab.cs.uno.edu/vroussev/csci4623-s26
```

➢ `tutorials/entropy-exercise.html`

You will:

1. Build a small disk image with four distinct regions
2. Run `ent` against each region
3. Plot the results and identify each region from entropy alone
4. Discover where entropy fails — and what to do about it

Block hashes & known-file filtering

Motivation

Scenario: 500 GB drive, looking for fragments of known files

- ◊ Whole-file hashing requires intact, allocated files
  - » deleted files → no directory entry → no file to hash
  - » fragmented files → blocks scattered across disk
  - » partial overwrites → file hash changes completely
- ◊ Block hashing operates on fixed-size chunks of raw disk
  - » works on any region: allocated, unallocated, fragmented
  - » each block is independently identified by its hash

Key insight: *known content* is still there even when the file is gone

From file hashes to block hashes

Whole-file hash

- one hash covers the entire file (or image)
  - change one byte → completely different hash

Block hash

- slice the image into fixed-size chunks (e.g. 4096 bytes)
  - hash each chunk independently
  - result: a table of (offset → hash) pairs

4096 bytes = one **extFS filesystem block** = one allocation unit

A block hash identifies content whether the block is allocated or not

Block size tradeoffs

512 bytes (sector level)

- finest granularity — matches physical sector size
- 1 TB image → ~2 billion hashes
- useful for MBR/GPT analysis, files smaller than one block

4096 bytes (filesystem block)

- matches ext block size, memory page size
- 1 TB image → ~250 million hashes
- practical sweet spot for most forensic workflows

Tradeoff: smaller blocks = finer granularity but exponentially more hashes

Critical: hash database and image scan must use the *same block size*

Known-file filtering workflow

Three-phase pipeline:

1. Build

» hash known reference files block-by-block → reference hash set

2. Scan

» hash the target image block-by-block → image hash set

3. Compare

» match image hashes against reference set

» matches = known content; non-matches = need analyst attention

Two modes:

◊ *negative filtering* (exclude ***KNOWN-GOOD***, e.g. OS files) and

◊ *positive filtering* (flag ***KNOWN-BAD***)

NSRL / Reference Data Set (RDS)

NIST National Software Reference Library (NSRL)

- curated hashes from known software
  - distributed as the Reference Data Set (RDS)
  - covers operating systems, applications, libraries

File-level vs. block-level

- NSRL distributes file-level hashes (MD5, SHA-1)
  - block-level sets must be built from source files
  - **hashdb** supports building block hash databases

Practical use

- negative filter: suppress known-good OS blocks
- positive filter: flag known-bad content
  - finds evidence even in deleted or fragmented files

Tools

Scripted approach

- ◊ dd + `sha256sum` in a loop
  - » transparent, slow, educational — you see exactly what happens

hashdb (Simson Garfinkel)

- ◊ purpose-built block hash database
  - » supports create, import, scan operations
  - » designed for scale: millions of hashes, disk-backed
  - » used by `bulk_extractor`

hfind (The Sleuth Kit)

- ◊ hash lookup against sorted hash databases
  - » supports NSRL format, integrates with TSK ecosystem

Connecting to TSK

Once block hashing identifies interesting offsets, TSK investigates:

- `blkstat` — check allocation status of a block address
  - is the block allocated or in unallocated space?
- `blkcat` — extract a specific block by address
  - visually inspect content, confirm the hash match
- `ifind -d` — find which inode owns that block
  - traces the block back to its file (even if deleted)
- `icat` — recover the full file from its inode
  - reconstructs the complete file for further analysis

Pipeline: block hash match → TSK offset lookup → file recovery

Limitations

- Compression, encryption, encoding defeat matching
  - identical content hashes differently after transformation
- Block alignment assumptions
  - if target data isn't aligned to block boundaries, matches fail
  - fragmented files may have partial blocks at the end
- Common-content false positives
  - blocks of all zeros, blocks of all `0xFF`
  - mitigation: exclude known-trivial hashes from the database
- Block size must match
  - database built at 4096 B won't match a scan at 512 B
- Hash function choice
  - MD5 for speed and NSRL compatibility
  - SHA-256 for stronger collision resistance

Hands-on exercise

```
git clone https://gitlab.cs.uno.edu/vroussev/csci4623-s26
```

- `tutorials/blockhash-exercise.html`

Tasks:

- ◊ Hash a disk image block-by-block using a scripted approach
- ◊ Build a reference hash set from known files
- ◊ Compare to identify known content — including a deleted file
- ◊ Investigate matched blocks with TSK (**blkstat**, **blkcat**, **ifind**, **icat**)
- ◊ Repeat the workflow using **hashdb** at scale